

Identifying Communities and Influencers in Social Networks with Machine Learning

Alex Klee
Virginia Tech CS 5805

1. Abstract

This paper presents a robust and scalable implementation of graph representation learning to identify community structures and influential actors within large-scale social networks. This project utilizes the Node2Vec framework to generate low-dimensional vector embeddings for nodes in the Ego-Facebook and Ego-Twitter datasets from the Stanford Large Network Dataset Collection (SNAP). This experiment addresses some of the computational challenges of processing large graphs, enabling the analysis of networks with over 1.7 million edges on standard hardware. Hyperparameters were optimized to yield the most accurate structural representations, finding that local neighborhood exploration is important for modeling these social networks. Experimental results reinforce the idea that these networks are hyper-partitioned structures composed of distinct communities connected by high centrality bridge nodes. These findings help to validate the effectiveness of unsupervised graph embeddings as a method for detecting meaningful social clusters and influencers.

2. Introduction and Problem Statement

Online social networks have recently grown into tools for far beyond communication and into platforms involving a massive spread of information and influences, with audiences all over the world. Yet, the flow of information and the underlying structure of these networks is quite complex and difficult to understand at such a large scale, on top of being obscured from users. This project aims to develop and evaluate machine learning methods for identifying specific communities and influential users within large social graphs. By using graph theoretic techniques alongside models trained on large scale graph based datasets, the project will explore scalable and accurate approaches to community and influencer detection.

Social networks can be represented using graph theory as graphs where nodes correspond to users or entities, and edges represent relationships, interactions, or shared interests. Understanding how these graphs are structured can give great insight on how ideas, opinions, and behaviors

spread online, which is crucial to understand in today's world. There are a few traditional non-ML techniques that have been used to understand how networks are organized: **modularity optimization** groups nodes into communities with higher density than between them, which can effectively identify clusters of closely connected users, defined as communities, while **betweenness centrality** measures how often a node lies on shortest paths, which can highlight individuals who may act as important distributors of information, defined as influencers. Although these statistical measures can provide decent insights on their own into local and global connectivity, without using any machine learning, they struggle with scalability and the ability to capture communities in large, real-world networks. Machine learning approaches such as **graph neural networks (GNNs)** and **unsupervised graph embeddings** offer a much more flexible and scalable framework for this type of research, and things like modularity optimization and betweenness centrality will provide serve as useful quantitative metrics.

The central problem this project aims to address is designing a scalable machine learning model that can learn meaningful structural representations of nodes and communities from large scale real-world graph data from the **Stanford Large Network Dataset Collection**. By leveraging machine learning tools such as GNNs and graph embeddings alongside measures such as modularity and betweenness centrality, this project seeks to contribute to research in the intersection between interpretable network theory and data driven graph learning along with general sociological research.

3. Background and Related Work

3.1. Current Practice and Limitations

Prediction tasks across large graphs with large amounts of nodes and edges are hugely important as well as challenging tasks for the field of network analysis, as they require algorithms to utilize a set of informative, discriminating, and independent features. Traditional, non-ML solutions for these type of prediction models involved hand picking domain specific features [1]. The main issue with

this conventional approach is that since these features are usually designed for specific tasks, they do not generalize well across different prediction tasks.

Alternative approaches have tried to learn these feature representations by creating and solving an optimization problem in an unsupervised, task-independent way [2]. These methods can be accurate, however, these approaches involve calculating eigendecompositions of matrices, which is expensive for large real-world networks. [3]

Recent advancements in this field of research have frequently been inspired by natural language processing (NLP) techniques like the skip-gram model, with more of a focus on preserving local neighborhoods of nodes. A couple examples, DeepWalk and LINE, have an efficient optimization approach doing this using stochastic gradient descent (SGD) [4]. However, these methods have been shown to be insensitive to diverse connection patterns in graphs, specifically failing to capture two common principles: **homophily**, where nodes belonging to the same community are typically closely connected and should be embedded together, and **structural equivalence**, where nodes that share similar functional roles are considered equivalent, even if they are far apart in the network. Real-world networks commonly have a mixture of both homophily and structural equivalence. Many of these existing strategies are effective in specific domains, but they all lack the flexibility necessary to fully explore diverse neighborhoods to capture both of these principles [2].

3.2. Significance

As mentioned earlier, the development of robust and flexible feature representations for large scale graphs is important to continue advancing prediction tasks in network analysis. There are a few different common prediction tasks in network analysis, a few examples include node classification (used to assign functional labels to proteins), link prediction (used for discovering novel gene interactions)[2], and community detection, which I will be focusing on.

The **node2vec** framework is proposed to overcome the discussed limitations of rigid neighborhood definitions by defining a more flexible notion of a node's neighborhood and implementing a biased random walk procedure [1]. The algorithm introduces tunable parameters for both return and in-out (p and q), meaning you can fully tune the search strategy to find the optimized random walk between two extremes: breadth first search (BFS)-like sampling (tends to capture structural equivalence) and depth first search (DFS)-like sampling (tends to capture homophily) [1].

A successful implementation of the **node2vec** algorithm should contribute to this area of research by:

1. **Learning Richer Representations:** The main improvement of **node2vec** over traditional algorithms lies

in the flexibility in exploring neighborhoods, which is the key to learning richer network representations.

2. **Improving Accuracy:** The method outperforms existing state-of-the-art techniques on multi-label classification tasks by up to 26.7% and on link prediction by up to 12.6% across diverse domains. [1]
3. **Ensuring Scalability:** The algorithm is designed for efficiency, since its major phases are all parallelizable and can scale to handle large networks with millions of nodes.
4. **Providing Robustness:** The features generated from this algorithm are shown to be more robust to random noise in network structure, such as noisy or missing edges.

Therefore, the ability of **node2vec** to flexibly learn these representations while simultaneously respecting both homophily and structural equivalence is a great advancement over traditional techniques, and ensures that the feature learning process is adaptable to the complex reality of modern network data.

4. Approach

This project will use datasets from the **Stanford Large Network Dataset Collection (SNAP)**, which has a large and diverse collection of large scale graph datasets representing real-world social networks including Facebook, Twitter, and YouTube. These datasets are well suited for graph based machine learning, and contains vast structural information and realistic connectivity patterns, which should help the model perform tasks like community and influencer identification.

Given the massive size and variety of the SNAP collection, this project will focus on a select subset of datasets that strike a good balance of scale and interpretability. While there are all kinds of networks in the SNAP collection, the goal of this project is focusing solely on data from social networks, where users are generally more meaningfully connected.

The core approach for my project will use **Node2Vec**, a graph representation machine learning method that generates node embeddings through biased random walks and skip-gram training. This technique has the potential of capturing both local neighborhoods as well as broader structural patterns, which is quite useful for better understanding node roles and community structure. The embeddings will be evaluated on community detection tasks using metrics such as modularity against the ground-truth community labels from the original dataset. My implementation utilizes the node2vec Python library, which uses optimized C-routines through Gensim for random walk generation and skip-gram training.

While the random walks generate the sequences of nodes, the actual feature learning in this algorithm is performed by the Skip-Gram with Negative Sampling (SGNS) algorithm. The objective here is to maximize the likelihood of preserving network neighborhoods in the d -dimensional feature space. Formally, for a graph $G = (V, E)$, we are aiming to learn a mapping function $f : V \rightarrow \mathbb{R}^d$ that maximizes the log-probability of observing a neighborhood $N_S(u)$ for a node u conditioned on its feature representation, given by the objective function:

$$\max_f \sum_{u \in V} \log P(N_S(u) | f(u))$$

This approach ensures scalability across datasets without the difficulty of complex custom neural network implementations.

4.1. Evaluation and Success Metrics

Modularity is the first metric I will use for community detection, and it measures the strength of division of a network into communities. It quantifies how much more densely connected nodes are within communities compared to a random network. The modularity Q is calculated as:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

where A_{ij} represents the edge weight between nodes i and j , k_i is the degree of node i , m is the total number of edges, and $\delta(c_i, c_j)$ is the Kronecker delta function. Modularity scores range from -0.5 to 1 , where higher values indicate stronger community structure.

Betweenness Centrality is another useful metric for community detection, and it identifies influential nodes by measuring how many shortest paths between node pairs pass through a given node. For a node v , the betweenness centrality $C_B(v)$ is given by:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

where σ_{st} is the total number of shortest paths from node s to node t , and $\sigma_{st}(v)$ is the number of those paths that pass through v . This metric is particularly useful for influencer identification tasks, since the nodes with high betweenness centrality act as bridges between communities and typically represent key connectors in social networks.

I will also track basic performance metrics such as **Training Time** and **Memory Usage** across datasets of varying size to help with scalability.

5. Experimental Setup and Results

To ensure optimal training, I will use a two-stage setup, first training a model using node2vec on a smaller dataset to optimize parameters, before moving on to a much larger dataset for large scale evaluation.

5.1. Dataset Description

The experiments utilize real-world social network data from the Stanford Large Network Dataset Collection (SNAP).

- **Prototyping and Optimization:** I will use 3 of the smaller datasets from the SNAP collection to find optimal hyperparameters for my model. These datasets (ego-Facebook, feather-lastfm-social, and congress-Twitter) all contain undirected social network graphs, ranging from 475 to 7,624 nodes and 13,289 to 88,234 edges.
- **Training:** I will use the Ego-Twitter dataset, a larger network consisting of approximately 81,306 nodes and 1.7 million edges. The dataset represents connections from the twitter platform.

5.2. Implementation and Hyperparameters

The core learning algorithm is implemented using the node2vec Python library. This approach allows for efficient parallelized random walk generation. The code used for training is available [here](#).

For the community detection task, the learned embeddings are clustered using the K-Means algorithm. Table 1 details the optimized hyperparameters I have selected for the prototype.

Parameter	Value	Definition
Dimensions (d)	64	The size of the vector
Walk Length (l)	30	Steps in one trip
Num Walks (r)	10	Walks per node
p (Return)	2.0	Return likelihood
q (In-out)	1.0	Move away likelihood
Clusters (k)	10	Forced # of communities

Table 1.

These parameters were deemed optimal after trial runs training the three smaller social circle datasets. The dimension of 64 was established first, as 64 is a standard choice here and performance tends to saturate once the dimension reaches around 100 [1]. To find the optimal value for p and q , I used an iterative process over the hyperparameter space of $p, q \in \{.25, .50, 1.0, 2.0, 4.0\}$, and averaged over the three datasets. All 3 datasets had relatively similar performance, across all 75 runs the modularity ranged from between 0.68 and 0.74. In this table, the rows represent the return parameter p and columns represent the in-out parameter q . The highest average modularity score is highlighted in bold, with $p = 2$ and $q = 1$, which I will use as my hyperparameters for training.

$p \backslash q$	0.25	0.5	1	2	4
0.25	0.6985	0.6961	0.7129	0.6271	0.6547
0.5	0.7217	0.7187	0.7088	0.6916	0.7003
1	0.7119	0.7167	0.7051	0.6823	0.7044
2	0.7119	0.7017	0.7245	0.7041	0.7138
4	0.7106	0.7187	0.7133	0.7078	0.7227

These values of p and q are not arbitrary, and reveal some information about the type of networks that the model is learning. A high return parameter greater than 1 ($p = 2$) effectively penalizes each walker for backtracking to the previously visited node. Setting $p > 1$ ensures that the model is less likely to sample an already-visited node in the following two steps, which is useful to prevent the model from overfitting to smaller 1-hop neighborhoods and encourages some exploration of the network.

The in-out parameter q controls the likelihood of moving farther away from the starting node. This parameter allows the search to differentiate between inward (local) and outward (global) nodes. q values less than 1 encourage depth first search behavior, which means the walk will visit nodes further away from the previous node. On the other hand, q values greater than 1 keep the walk more local and approximate breadth first search behavior, which is effective for characterizing structural equivalence. [1] This means our optimal value of $q=1$ indicates that the model favors the standard, uniform random walk. Therefore, the maximum modularity score is achieved by relying primarily on the graph’s natural structure.

To select the optimal number of clusters (k), modularity scores were computed across $k \in \{3, 5, 10, 15, 25\}$. The modularity consistently increased up to $k = 10$, but generally began decreasing beyond $k = 10$. With this information I chose $k = 10$ as my desired number of communities, since it seemed to be a good middle ground and highly generalizable across the different datasets.

5.3. Prototype Validation (Facebook)

Once the optimized parameters were established, the Node2Vec model was trained on the Ego-Facebook dataset to generate 64-dimension embeddings for all 4,039 nodes. After training was complete, dimensionality reduction was performed to visualize the resulting vector space in 2D. The modularity score for the graph is calculated, along with the top 5 most influential users, using betweenness centrality.

5.3.1 Community Structure and Visualization

Using the k-means algorithm ($k = 10$), the graph can be partitioned into distinct communities. To represent these in a visualization, a color scale is used to distinguish between communities. The quality of this partition was evaluated using the modularity metric.

This model achieved a modularity score of **0.7775**. In social network analysis, modularity values above 0.3 are considered a significant community structure. A score above .7 is a clear indicator that the Node2Vec embeddings successfully encoded the underlying graph topology and achieved highly accurate unsupervised clustering. This score also shows that these communities were highly distinct, and largely interacted within themselves rather than across communities. [5]

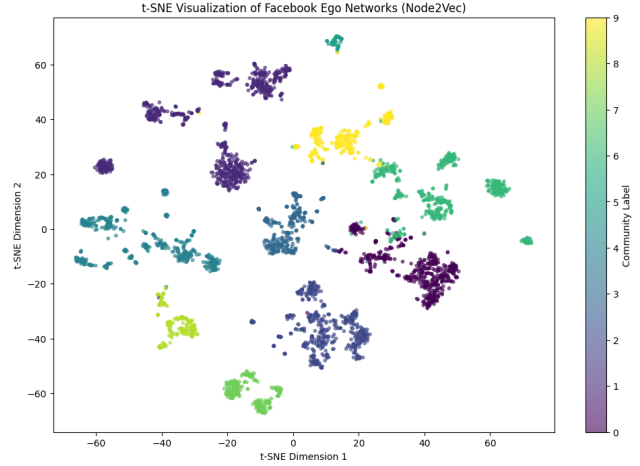


Figure 1.

t-Distributed Stochastic Neighbor Embedding (t-SNE) and matplotlib were used to visualize the latent representations learned by the Node2Vec model. This dimensionality reduction technique projects the 64-dimensional embedding vectors into a 2-dimensional plane for visual analysis. Points that are physically closer together in the plot share similar structural roles or network neighborhoods in the original graph. It is also easy to see the distinct color coded clusters in the visualization, indicating that the Node2Vec model successfully encoded the graph’s community structure into the vector space.

5.3.2 Influencer Identification

While the clustering calculation reveals groups, it does not identify the key users within these networks. To find these influential users, the betweenness centrality was calculated for the graph. This metric computes the fraction of all shortest paths in the network that pass through a specific node, which ends up effectively finding the bridges that control information flow between communities. Table 2 presents the top 10 influencers identified by the model.

The centrality distribution reveals some interesting information about the hierarchy within the network. Node 107 has a score of 0.497, which means that nearly 50% of all efficient paths in the network pass through this single user, clearly indicating a very strong influence on the network

Table 2.

Rank	Node ID	Centrality Score
1	107	0.4971
2	1684	0.3458
3	1912	0.2359
4	3437	0.1875
5	0	0.1573
6	1085	0.1311
7	698	0.0926
8	58	0.0919
9	567	0.0759
10	428	0.0616

from this node. Following the primary hub of node 107, nodes such as 1684 and 1912 serve as important bridges, also connecting large communities. These calculations reinforce the idea that a small minority of users bridge the vast majority of connections and information flow in social networks.

5.4. Scaling to Large-Scale Networks (Twitter)

Following the successful validation of the model and hyperparameters on the Facebook prototype, the training pipeline was scaled to the Ego-Twitter dataset. This dataset represents a significantly more complex challenge, containing 81,306 nodes and over 1.7 million edges, which is nearly 20 times the size of the Facebook network.

5.4.1 Computational Challenges and Optimization

Scaling Node2Vec to this magnitude introduced a few computational bottlenecks. The main challenge was storing the pre-computed transition probabilities for 1.7 million edges, which generally exceeded RAM capabilities.

To resolve this, instead of loading the entire probability matrix into volatile memory, the transition probabilities were computed and sent to a temporary directory on the disk. This allowed the parallelized random walk generation to access probability data via demand paging, which kept the RAM usage at an acceptable level. This did slightly slow down training, but it enabled the processing of the massive graph without any sort of high performance computing access.

5.4.2 Community Structure and Visualization

The trained embeddings were clustered using K-Means ($k = 10$) to identify communities within this Twitter network. The model achieved a Modularity score of **0.7428**, again showing that the graph topology was successfully found even on this large scale graph. Again, a high score exceeding 0.7 indicates a **hyper-partitioned** structure. This suggests that the network is composed of highly distinct and

tight-knit communities that rarely interact with other communities.

Visualization of the entire 81,000-node network was computationally difficult and also resulted in a lot of clutter in the graph. To address this problem, a random sample of 10,000 nodes was plotted. The resulting graph (Figure 2) shows distinct clustering that was found, indicating that the model generalized well to the Twitter topology.

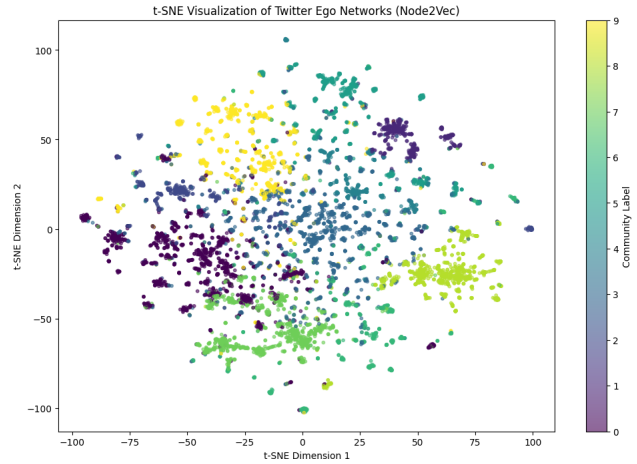


Figure 2.

5.4.3 Influencer Identification

Betweenness centrality was calculated again to identify key information brokers. Due to the graph's size, betweenness centrality was approximated using random sampling of $k = 1000$ source nodes. This random sampling method has been shown to reliably and accurately approximate this metric [6].

Table 3.

Rank	Node ID	Centrality Score
1	813286	0.068484
2	115485051	0.049307
3	3359851	0.047878
4	783214	0.032935
5	7861312	0.027095
6	15846407	0.026067
7	17093617	0.023402
8	14230524	0.023166
9	40981798	0.022269
10	258145892	0.021413

The raw centrality scores for the Twitter dataset are considerably lower than those in the Facebook dataset. This is largely just due to the network size, since betweenness centrality measures the percentage of total network paths passing through a node, and in a graph of 81,000 nodes, there

would be 6.6 billion potential paths. Despite the lower raw numbers, the top-ranked nodes still control huge amounts of information flow. The top influencer (node 813286) has a centrality score of 6.8%, which means almost 500 million of these shortest paths must pass through this single node to reach their destination.

The experiment on the Twitter dataset shows that the trained embeddings successfully encoded a complex and realistic topology characterized by high modularity ($Q = 0.74$) at large scale. The analysis reveals a network structure consisting of distinct communities connected largely through a number of bridges / influencers.

5.4.4 Code Availability

The implementation of the scalable Node2Vec pipeline, with scripts for data preprocessing and hyperparameter customization, is open-source and available at <https://github.com/abklee/socialnetworkanalysis>.

6. Conclusion

This project focused on a practical application and scalable implementation of the Node2Vec framework to analyze community structures and influencers within social networks. Social network datasets of varying size from the Stanford Large Network Dataset Collection (SNAP) were used to test the effectiveness of graph representation learning in preserving topological features such as community structure. This work contributes a flexible and scalable training pipeline capable of training on low-resource systems. With memory management, the existing Node2Vec library was shown to successfully scale to process the Ego-Twitter dataset with over 81,000 nodes and 1.7 million edges on standard hardware. These experimental results also reinforce existing findings related to social network topology. The hyperparameter optimization process helped show that forcing the random walks to explore local neighborhoods rather than backtracking yields richer structural representations. Also, the high modularity scores obtained in both the Facebook and Twitter experiments align with the idea that these networks are hyper-partitioned. Finally, the betweenness centrality was calculated to validate the model's ability to identify 'bridge' nodes (influencers) within these partitions. The detection of these influencers who control significant information flow shows how this pipeline can be a useful tool for extracting insights from large and complex graphs. In conclusion, this project provides a robust and simple implementation of the Node2Vec algorithm for applying graph embeddings to social network analysis, confirming the effectiveness of the Node2Vec algorithm for identifying communities and influencers.

References

- [1] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 855–864, 2016. 1, 2, 3, 4
- [2] A. Karateev, D. Li, G. Gaume, and H. Yamada, "A comprehensive review of community detection in graphs," 2023. 2
- [3] D. Jin, Z. Yu, P. Jiao, S. Pan, D. He, J. Wu, P. S. Yu, and W. Zhang, "A survey of community detection approaches: From statistical modeling to deep learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 2, pp. 1149–1170, 2021. 2
- [4] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 701–710, ACM, 2014. 2
- [5] M. E. J. Newman, "Modularity and community structure in networks," *Proceedings of the National Academy of Sciences*, vol. 103, no. 23, pp. 8577–8582, 2006. 4
- [6] M. Riondato and E. M. Kornaropoulos, "Fast approximation of betweenness centrality through sampling," in *Proceedings of the 7th ACM International Conference on Web Search and Data Mining*, WSDM '14, pp. 413–422, ACM, 2014. 5